Все о Компьютерах · Подписаться на публикацию

Вы читаете бесплатно через Gopi C K's Friend Link. Станьте участником , чтобы получить доступ к лучшим материалам Medium.

История только для участников Избранное

ТЕХНОЛОГИЯ + Кодирование + Программирование +
Разработка программного обеспечения + Серверная часть +

Хватит следовать инструкциям: вместо этого создайте эти 5 бэкенд-проектов

Создайте эти 5 бэкенд-проектов, чтобы освоить API, базы данных, проектирование систем и решение проблем на уровне производства.

18 минут чтения · 4 дня назад

Гопи Си Кей Подписаться

Слушай Поделиться

Большинство разработчиков могут, следуя инструкциям, создать работающий бэкенд. Они могут без особого труда создавать REST API, подключать базу данных, реализовывать аутентификацию и возвращать ответы в формате JSON. Настоящая сложность начинается, когда приложению нужно обрабатывать повторяющиеся запросы, токены с истекшим сроком действия, медленные запросы к базе данных, непредвиденные сбои и действия пользователей, которых разработчик не ожидал.

Именно здесь становится очевидной разница между знанием технологии и пониманием серверной разработки. Учебное пособие обычно ведет вас к рабочему результату, в то время как реальный проект заставляет принимать решения, когда нет единственно верного ответа. Вы должны решить, где

должна находиться бизнес-логика, как следует хранить данные, что происходит, если запрос не выполняется до конца, и как сделать так, чтобы действия одного пользователя не влияли на информацию другого пользователя.

> *Если вы не являетесь подписчиком Medium? Прочитайте здесь*

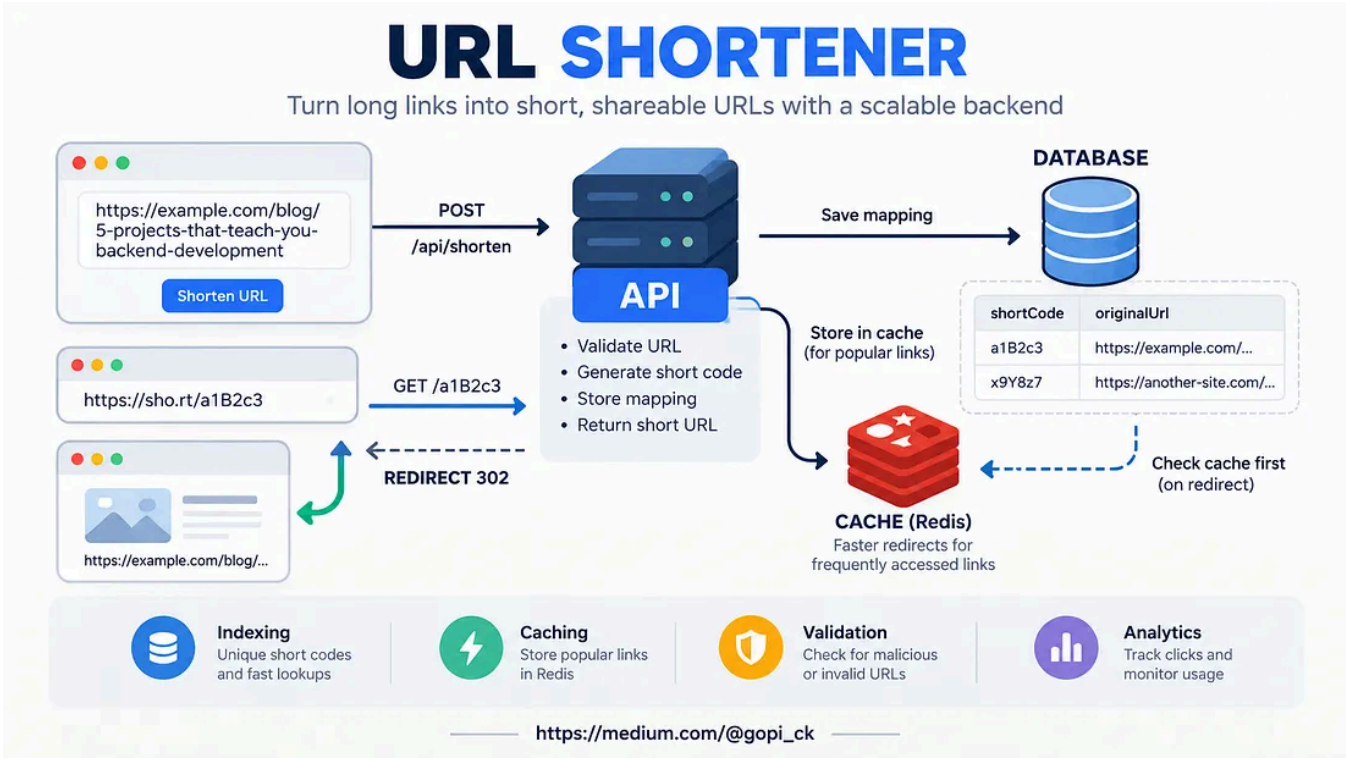


Сгенерированное Искусственным интеллектом Изображение

Лучший способ развить эти навыки — не обязательно создавать что-то масштабное. Нужно создавать проекты, которые помогут вам столкнуться с проблемами, с которыми бэкенд-разработчики сталкиваются каждый день. Тщательно продуманный...

Вот пять проектов, которые стоит реализовать, если вы хотите не просто копировать код, а начать мыслить как бэкенд-разработчик. У каждого проекта есть понятная отправная точка, набор все более сложных задач и уроки, которые можно применить непосредственно в производственных приложениях и на технических собеседованиях.

1. Создайте сервис сокращения URL, как Bitly



Сокращение URL-адресов выглядит обманчиво простым. Пользователь отправляет длинный URL-адрес, сервер генерирует короткий код, и все, кто переходит по этому коду, перенаправляются на исходный сайт. Можно создать базовую версию с одной конечной точкой и базой данных, но интересные инженерные решения появляются, когда нужно учитывать коллизии, вредоносные ссылки, повторные запросы, аналитику и структуру трафика.

Это отличный первый проект, потому что он начинается с небольшой функции и постепенно знакомит с несколькими фундаментальными концепциями бэкенда. Вместо того чтобы создавать десятки несвязанных между собой CRUD-интерфейсов, вы можете сосредоточиться на понимании того, как один запрос проходит через ваше приложение, как база данных обрабатывает этот запрос и что меняется, когда на ту же конечную точку приходится значительно больший трафик.

Что нужно создать в первую очередь?

Начните с двух конечных точек: одна будет создавать короткий URL, а другая — перенаправлять посетителей на исходный сайт. Сохраняйте каждое сопоставление с уникальным коротким кодом, исходным URL, временем создания и, при необходимости, датой истечения срока действия. Вы можете использовать Node. js и Express с MongoDB или Java с Spring Boot и PostgreSQL; основные принципы проектирования остаются неизменными.

Короткий код должен генерироваться таким образом, чтобы можно было справиться с коллизиями. Один из вариантов — сгенерировать достаточно случайный токен и применить уникальное ограничение базы данных, повторяя попытку в случае коллизии. Другой вариант — присвоить уникальный числовой идентификатор и закодировать его с помощью кодировки Base62, которая представляет числа с помощью 62 символов. Второй подход позволяет создавать компактные идентификаторы, но последовательные идентификаторы легче поддаются перечислению, а распределенное распределение идентификаторов требует дополнительного внимания.

Вот упрощенный пример Express для конечной точки перенаправления:

```
app.get("/:code", async (req, res, next) => {
  попробуй {
    const link = await Url.findOne({
 shortCode: req.params.code
 });
    if (!link) {
      return res.status(404).send("Ссылка не найдена");
 }
    if (link.expiresAt && link.expiresAt <= new Date()) {
      return res.status(410).send("Срок действия ссылки истёк");
 }
    return res.redirect(302, link.originalUrl);
 } catch (error) {
 next(error);
 }
});
```

В этом примере показан базовый процесс, но намеренно не учитываются некоторые проблемы, возникающие при работе с реальными данными. Приложение должно проверять URL-адреса при создании, при необходимости ограничивать использование неподдерживаемых протоколов, обеспечивать уникальность на уровне базы данных и применять соответствующие ограничения скорости. Постоянное перенаправление, например HTTP 301, может кэшироваться клиентами, что может усложнить изменение адресата и аналитику кликов, поэтому на ранних этапах разработки часто удобнее использовать временное перенаправление.

Чему вас научит этот проект?

Первый важный урок заключается в том, что структура базы данных влияет на поведение приложения, а не только на хранение данных. Уникальный индекс в `shortCode` не позволяет двум записям иметь одинаковый идентификатор, даже если несколько запросов одновременно пытаются создать одну и ту же запись. Индекс также помогает базе данных эффективно находить сопоставления, не просматривая каждый документ по мере роста коллекции.

Второй урок — кэширование. Популярные ссылки могут запрашиваться неоднократно, поэтому можно использовать Redis для хранения недавно использованных сопоставлений. При поступлении запроса приложение сначала проверяет кэш, а в случае промаха — обращается к базе данных. Однако кэширование приводит к необходимости устанавливать срок действия, решать проблемы с аннулированием и допускать возможность предоставления устаревшей информации. Кэширование полезно, когда это оправдано шаблонами доступа, а не просто потому, что Redis используется в популярных бэкенд-архитектурах.

Затем вы можете добавить аналитику кликов, не замедляя процесс перенаправления. Вместо того чтобы выполнять ресурсоемкую аналитическую работу перед отправкой ответа, приложение может записать событие для асинхронной обработки. В связи с этим возникает новый вопрос: что произойдет, если событие будет потеряно, обработано дважды или с задержкой?

Наконец, защитите сервис от злоупотреблений. Сокращатели общедоступных URL-адресов могут использоваться для рассылки спама, фишинга и автоматического трафика. Ограничение скорости, проверка назначения, мониторинг и механизмы оповещения о злоупотреблениях становятся частью функционала бэкенда, а не дополнительными опциями.

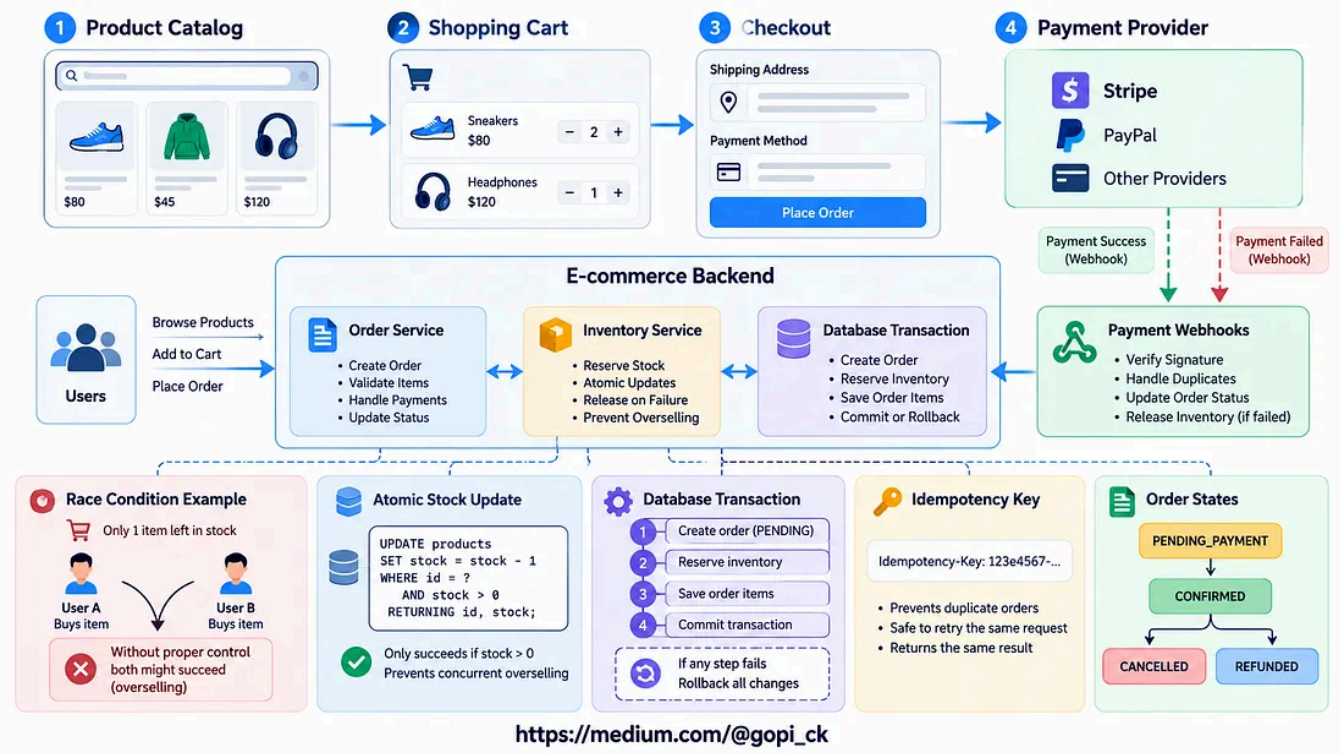
> *Что делает этот проект ценным: он учит работе с запросами, моделированию данных, индексированию, кэшированию, перенаправлению HTTP-запросов, проверке входных данных и компромиссам в производительности на примере одной функции, которую легко понять, но сложно довести до совершенства.*
> ***Тренируйтесь Здесь***

15 минут на изучение принципов работы системы сокращения URL

Разработайте масштабируемую систему сокращения URL-адресов с эффективными перенаправлениями, уникальными...

medium.com

2. Создайте серверную часть для электронной коммерции с управлением запасами



Приложение для электронной коммерции — один из лучших способов понять, почему разработка бэкенда — это гораздо больше, чем создание API. Конечная точка для отображения списка товаров проста, а создание заказа на первый взгляд может показаться обычной вставкой данных в базу. Но что произойдет, если два покупателя попытаются приобрести последний доступный товар в одно и то же время? Что, если оплата пройдет успешно, но приложение выйдет из строя до того, как заказ будет оформлен? Что, если покупатель несколько раз нажмет кнопку оформления заказа, потому что страница зависла?

В этих сценариях рассматриваются некоторые из наиболее важных концепций бэкенд-разработки: транзакции, управление параллелизмом, идемпотентность, управление состоянием и надежная интеграция с внешними сервисами. Они также показывают, почему набор отдельных функциональных конечных точек не является автоматически готовым приложением.

Начните с бизнес-процесса

Создайте бэкенд, поддерживающий работу с пользователями, товарами, корзинами, заказами, складскими запасами и платежами. Первоначальный процесс должен быть простым: клиент добавляет товары в корзину, оформляет заказ и получает подтверждение после обработки платежа. У каждого объекта должна быть четкая цель, а взаимосвязи между ними должны отражать бизнес-правила.

Для этого проекта особенно хорошо подходит PostgreSQL, поскольку реляционные ограничения и транзакции предоставляют надежные инструменты для обеспечения согласованности связанных записей. Например, в заказе должны сохраняться цена и количество, согласованные при оформлении, а не зависеть от текущей цены товара. В противном случае изменение цены товара завтра может непреднамеренно повлиять на историческую стоимость вчерашнего заказа.

Упрощенное обновление инвентаря может выглядеть так:

```
UPDATE products
SET stock = stock - 1
WHERE id = $1
 И запас > 0,
ВОЗВРАЩАЮЩИЙ идентификатор, запас;
```

Приложение должно проверить, вернул ли запрос строку. Если да, то резервирование товара прошло успешно; если нет, значит, товар был недоступен или его не существовало. Поскольку условие и обновление выполняются как один оператор базы данных, одновременные запросы не могут одновременно успешно уменьшить количество единиц товара на складе с единицы до нуля.

Это надежнее, чем считывать данные о наличии товара в памяти приложения, проверять, есть ли товар в наличии, а затем отправлять отдельное обновление. Два запроса могут считать одно и то же значение до того, как оно будет обновлено, что приведет к ошибке при продаже.

Научитесь управлять транзакциями и сбоями при оплате

Транзакция в базе данных позволяет одновременно выполнить или отменить связанные операции с базой данных. Предположим, для оформления заказа необходимо создать запись о заказе, добавить позиции и зарезервировать товар. Если одна из операций завершится неудачей, вам может потребоваться отменить другие изменения, чтобы не допустить несогласованности в системе.

Однако транзакции в базе данных не могут автоматически отменить успешное списание средств внешним платежным провайдером. Обработка платежей выходит за рамки системы, а значит, при проектировании необходимо учитывать возможность частичного сбоя. Более реалистичный рабочий процесс предполагает создание заказа в состоянии ожидания, резервирование товара, запрос оплаты и обновление заказа на основе подтвержденного результата оплаты. Если платеж не проходит, приложение может отменить резервирование в соответствии со своей политикой управления запасами.

Именно здесь важна идемпотентность. Идемпотентная операция может быть повторена без непреднамеренного дополнительного эффекта. Если клиент повторно отправляет запрос на оформление заказа, потому что первый ответ так и не пришел, серверная часть не должна создавать несколько заказов или взимать с клиента плату дважды. Это можно реализовать с помощью ключа идемпотентности, связанного с клиентом и попыткой оформления заказа, подкрепленного ограничением базы данных и сохраненным результатом операции.

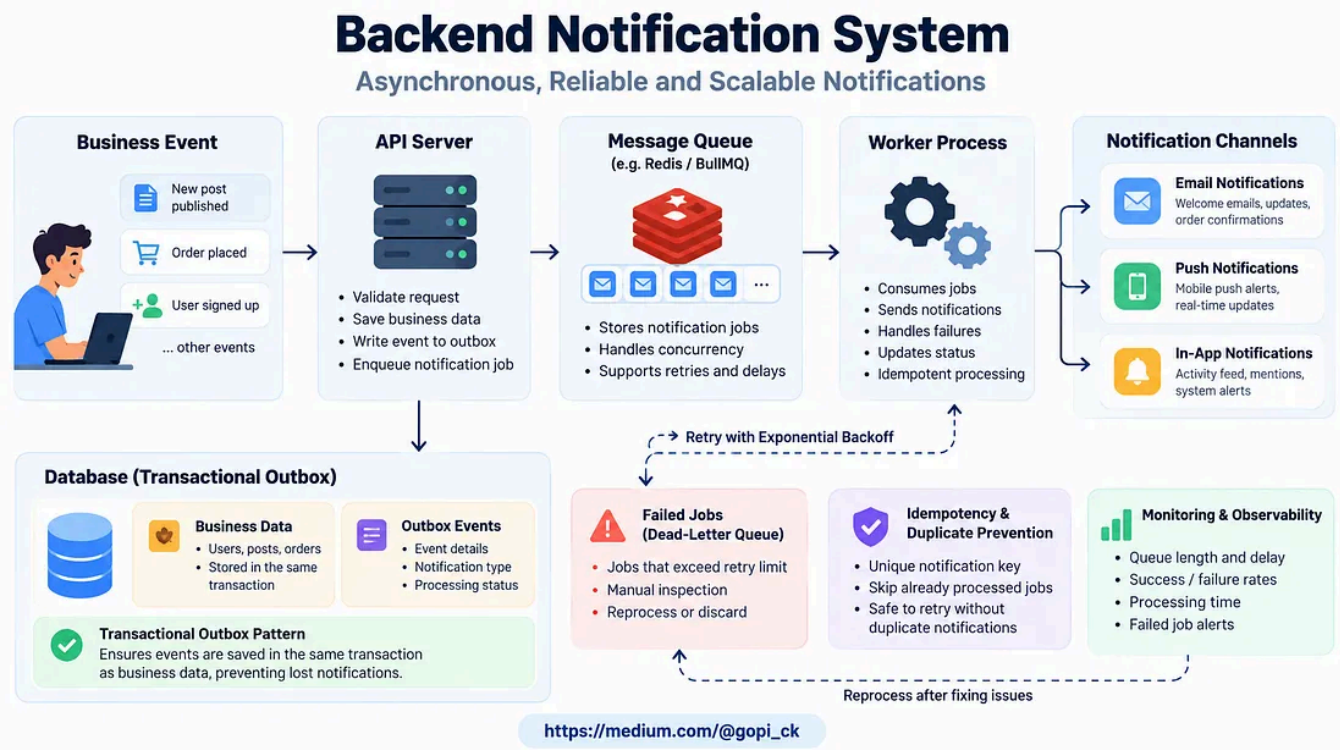
Превратите его в проект, готовый к запуску

После того как базовая система оформления заказов заработает, добавьте переходы между статусами заказа, вебхуки для оплаты, правила отмены, резервирование товаров с указанием срока действия и фоновое задание для обработки заказов, застрявших в статусе «ожидает». Проверяйте подписи вебхуков и разрабатывайте обработчики, допускающие повторную доставку, поскольку внешние сервисы могут повторно отправлять уведомления.

Не рассматривайте статус заказа как неограниченную строку, которую может изменить любая конечная точка. Определите допустимые переходы, такие как `PENDING_PAYMENT`, `CONFIRMED`, `CANCELLED` и `REFUNDED`, и обеспечьте соблюдение правил на бизнес-уровне. Заказ, по которому уже был произведен возврат средств, не должен автоматически возвращаться в статус «подтвержден» из-за того, что старый вебхук пришел с опозданием.

> *Что делает этот проект ценным: вы узнаете, что корректность так же важна, как и успешные ответы. Транзакции, управление параллелизмом, идемпотентность и явные бизнес-правила полезны не только в электронной коммерции, но и на платформах бронирования, в системах подписки, финансовых процессах и при бронировании билетов.*
> ***Тренируйтесь Здесь***

3. Создайте систему уведомлений с помощью фоновых задач



Рассмотрим ситуацию, когда пользователь публикует пост, а платформе нужно уведомить об этом сотни подписчиков. Серверная часть должна сохранить пост, определить, кого нужно уведомить, создать записи для уведомлений и, возможно, отправить электронные письма или push-уведомления. Выполнение каждой задачи в рамках исходного HTTP-запроса может привести к замедлению и снижению надежности ответа, особенно если внешним сервисам требуется время для ответа.

A notification system teaches you to separate work that must happen immediately from work that can happen asynchronously. This is a natural introduction to queues, worker processes, retries, failure handling, and eventual consistency. These

concepts are especially useful in applications involving user activity, scheduled reports, media processing, and large-scale email delivery.

Separate the request from the work

Start by creating an endpoint that accepts an event, such as a new post or a completed order. The API validates the request and stores the necessary business data. It then arranges for notification processing to happen separately, allowing the user-facing request to finish without waiting for every notification to be delivered.

For a small project, a Redis-backed queue such as BullMQ can help you implement this pattern in Node.js. The API enqueues jobs, while a separate worker process consumes them and performs the actual notification work. For larger systems or different operational requirements, a message broker such as RabbitMQ or Kafka may be appropriate, although these technologies have different delivery models and should not be treated as interchangeable.

A simplified example using BullMQ looks like this:

```
import { Queue, Worker } from "bullmq";
const notificationQueue = new Queue("notifications", {
  connection: {
    host: "127.0.0.1",
    port: 6379
  }
});
// Called after the relevant business event is recorded.
await notificationQueue.add("send-email", {
  userId: "user-123",
  template: "welcome"
});
const worker = new Worker(
  "notifications",
  async (job) => {
    if (job.name === "send-email") {
      await sendWelcomeEmail(job.data.userId);
    }
  },
  {
    connection: {
      host: "127.0.0.1",
      port: 6379
    }
  }
);
```

The code illustrates the separation between producing a job and processing it. A real implementation would also configure connection security, sensible retry and backoff policies, job retention, error monitoring, and appropriate concurrency. The email function should resolve only when delivery has been accepted by the relevant provider, not merely when a request has been constructed.

Understand retries, duplicates, and failure recovery

Background processing does not eliminate failures; it changes how you handle them. A worker might successfully send an email and crash before acknowledging the job. Depending on the queue's behavior, the job could run again, resulting in a duplicate email. A temporary provider outage could also cause repeated failures, while an invalid recipient might never become deliverable.

This is why retry policies should distinguish transient failures from permanent ones. A network timeout might justify another attempt after a delay, whereas a malformed email address usually requires intervention or correction. Exponential backoff can spread retries over increasing intervals instead of repeatedly hitting a failing service. After a configured number of attempts, a job may be moved to a failed-job collection or dead-letter queue for investigation.

Another important concept is idempotent processing. You might create a unique notification record for a specific combination of recipient, event, and notification type, preventing duplicate records when an event is processed more than once. For email delivery, complete exactly-once behavior is difficult to guarantee across independent systems; design around duplicate tolerance rather than assuming a job executes precisely once.

There is also a subtle failure window between saving business data and publishing a message. If an order is committed to the database but the application crashes before the corresponding job is enqueued, the order exists without a notification. The transactional outbox pattern addresses this by storing an event in the same database transaction as the business change. A separate publisher reads the outbox and sends events to the queue, with duplicate-safe processing for events that are published more than once.

What should you add next?

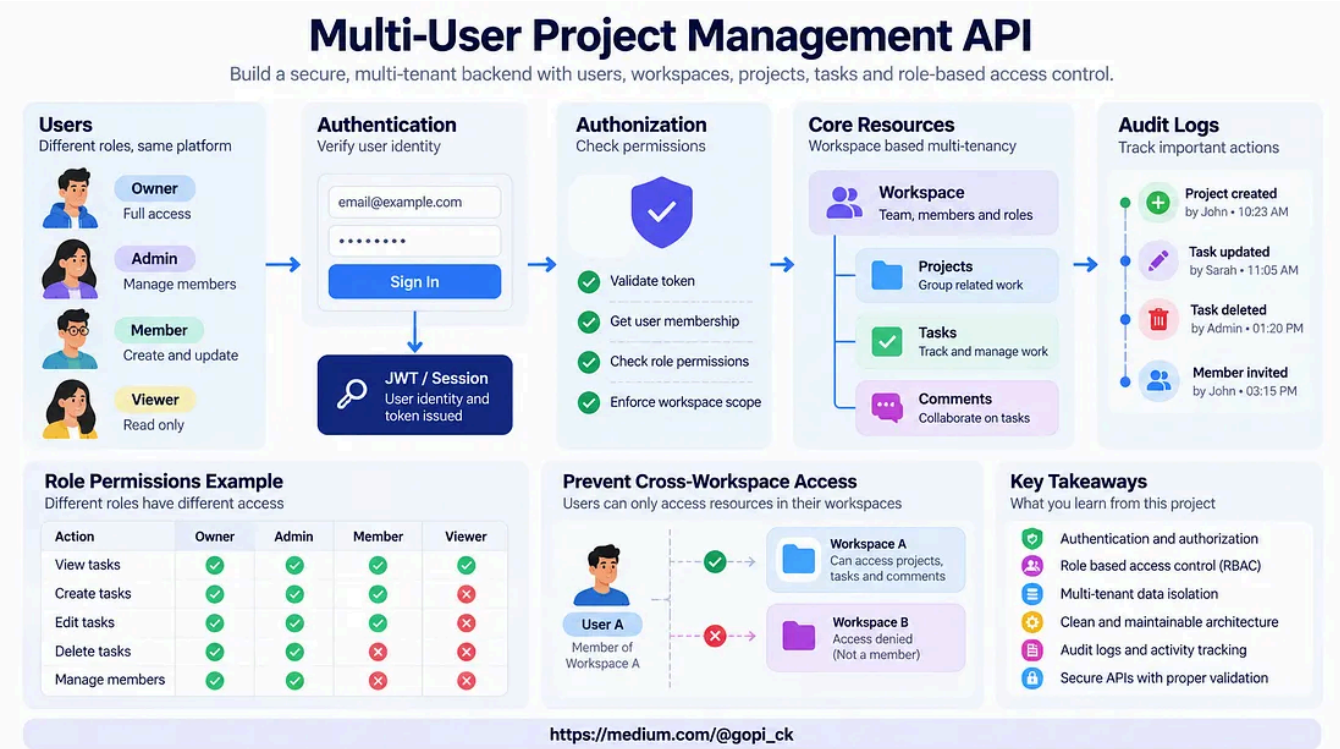
Build notification preferences, email and in-app delivery channels, scheduled reminders, retry dashboards, and a worker that processes multiple jobs concurrently. Track job states and measure queue delay, processing duration, and

failure rates. These metrics tell you whether notifications are simply slow or whether the entire worker pipeline is falling behind.

Be careful not to create a separate microservice for every small feature just to make the architecture look sophisticated. A queue and worker can run alongside a monolithic backend and still provide meaningful isolation. Additional services are useful when deployment independence, scaling requirements, or team boundaries justify the operational cost.

> *What makes this project valuable: it teaches asynchronous architecture, queue semantics, idempotency, retries, observability, and recovery from partial failures. These are the problems that distinguish a background task that works locally from one that can be trusted in a deployed application.*
> ***Practice Here***

4. Build a Multi-User Project Management API



A project management API may seem less technically challenging than an e-commerce platform, but it exposes a problem that developers frequently underestimate: authorization. Creating users, projects, tasks, and comments is relatively easy. Ensuring that every user can access only the resources they are

allowed to see is much harder, especially when the same application supports personal workspaces, shared projects, administrators, and invited collaborators.

This project is particularly valuable because it forces you to think beyond authentication. Authentication establishes who a user is; authorization determines what that user is allowed to do. A valid JWT does not mean its owner can update every project in the database, and hiding a button in the frontend does not prevent a malicious client from calling the corresponding API directly.

Design permissions before writing endpoints

Create users, workspaces, memberships, projects, tasks, and comments. Each workspace can contain multiple projects, and each project belongs to a workspace. Membership records can define roles such as owner, administrator, member, or viewer, with clearly documented permissions for each role.

For example, a viewer may read tasks but cannot edit them, while a member can update assigned work. An administrator might invite collaborators, and an owner could manage workspace-wide settings. The important part is to define these rules in the backend rather than scattering ad hoc role checks throughout route handlers.

A dangerous implementation might retrieve a task solely by its ID and immediately return it. An attacker who learns or guesses another task's identifier could access data belonging to a different workspace. This is a form of broken object-level authorization, a common API security risk.

A safer query scopes the requested resource to the authenticated user's access:

```
const task = await Task.findOne({
  _id: req.params.taskId,
  workspaceId: { $in: authorizedWorkspaceIds }
});
if (!task) {
  return res.status(404).json({
    message: "Task not found"
  });
}
```

Here, `authorizedWorkspaceIds` must be derived from trusted server-side membership checks, not supplied by the client. This query is an illustrative MongoDB pattern; the

correct implementation depends on your data model and authorization policy. In more complex systems, the server should also verify that the user has permission to perform the specific requested action, not just permission to access the workspace.

Learn clean architecture through real requirements

As the project grows, route handlers can become crowded with validation, permission checks, database queries, and business rules. You can improve maintainability by separating responsibilities into layers: controllers handle HTTP input and output, services implement business rules, and repositories or data-access modules encapsulate database operations where that abstraction is useful.

For example, a `TaskService` might verify membership, check whether the requested status transition is valid, and record the change. The controller should not independently implement these rules for every endpoint. Centralizing business logic makes it easier to test and reduces the risk that one route accidentally allows an operation that another route correctly rejects.

Do not turn this into an exercise in creating dozens of interfaces and folders. The goal is to make changes safer, not to maximize architectural complexity. A straightforward service layer is often sufficient for a portfolio project, and additional abstractions should solve a problem you can explain.

Add features that expose deeper backend concepts

Implement pagination, filtering, search, audit logs, and optimistic concurrency control. Pagination teaches you to avoid returning an entire workspace's task history in one response. Audit logs teach you to distinguish the current state of a task from the sequence of actions that produced it. Optimistic concurrency control can prevent one collaborator from silently overwriting another person's recent changes.

You can also introduce refresh-token rotation, session revocation, request validation, and rate limiting. Write integration tests that deliberately attempt cross-workspace access, edit resources using insufficient permissions, and reuse expired credentials. These tests are more valuable than simply checking that the happy path returns HTTP 200.

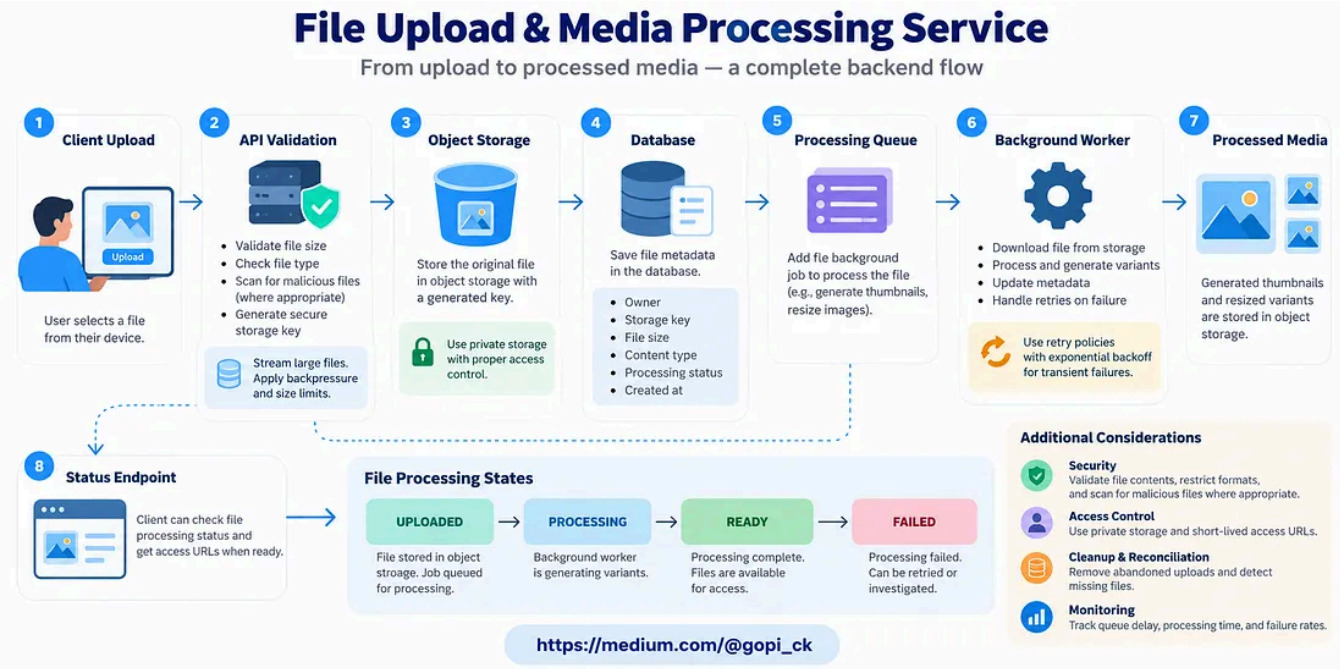
> *What makes this project valuable: it develops your understanding of API security, authorization, maintainable architecture, testing, and multi-tenant data isolation. Those skills are directly relevant to collaboration tools, SaaS products, internal business platforms, and many full-stack applications.*
> ***Practice Here***

10 REST API Design Principles Every Backend Developer Should Follow

From clean endpoints and HTTP semantics to pagination, idempotency, versioning, and rate limiting — the practical rules...

medium.com

5. Build a File Upload and Media Processing Service



File uploads are common in social networks, learning platforms, document management systems, and collaboration tools. A basic upload endpoint might accept an image and save it to disk, but that implementation quickly becomes limiting when files are large, users upload multiple files simultaneously, or the application needs thumbnails, previews, and download permissions.

This project introduces a different set of backend challenges: streaming, object storage, asynchronous processing, resource limits, access control, and secure handling of untrusted files. It is particularly useful if you want to build applications that support profile pictures, posts, documents, or other user-generated content.

Design the upload pipeline

Start with an endpoint that accepts an uploaded file, validates its size and permitted format, and stores it in object storage such as Amazon S3 or a compatible service. Store the file's metadata in your database, including its owner, storage key, size, detected content type, creation timestamp, and processing status.

For larger files, consider uploading directly from the client to object storage using a short-lived presigned URL. Your backend authorizes the operation and generates the URL, while the client transfers the file without routing every byte through the application server. This can reduce pressure on the API server, although the backend must still enforce ownership, size limits, allowed content types, and appropriate storage permissions.

Avoid treating the filename or MIME type provided by the client as proof that a file is safe. Validate the file's actual contents where possible, generate your own storage keys, restrict accepted formats, and consider malware scanning when the application handles potentially dangerous files. Publicly accessible uploads can also expose sensitive information, so private objects and controlled download access are often the safer defaults.

Process files outside the request

Suppose a user uploads a high-resolution image for a social media post. The application might need to generate a thumbnail, resize the image for different screen sizes, and extract metadata. Performing all these operations before returning the upload response can consume CPU and memory, making the API less responsive under concurrent uploads.

Instead, save the original file and enqueue a processing job. A worker retrieves the object, validates or decodes it using an appropriate library, generates the required variants, and updates the database when processing completes. The client can poll a status endpoint or receive a notification when the asset becomes available.

A simple state model might include `UPLOADED`, `PROCESSING`, `READY`, and `FAILED`. These states help the frontend communicate progress without pretending the file is immediately usable. They also make failures easier to investigate because you can distinguish an upload that never completed from one that was stored successfully but failed during thumbnail generation.

Handle operational problems deliberately

Large files can exhaust memory if your application buffers every upload entirely before processing it. Streaming data where appropriate, enforcing upload limits, and applying backpressure can reduce this risk. You should also configure timeouts and resource limits for image decoders and other processing tools, because untrusted files may be malformed or deliberately designed to consume excessive resources.

Design cleanup jobs for incomplete uploads and abandoned objects. If the database records a file that does not exist in storage, the application needs a recovery strategy; similarly, objects left behind after failed database operations can accumulate storage costs. Since object storage and a relational database generally do not share one transaction, reliable cleanup and reconciliation are important parts of the design.

> *What makes this project valuable: you learn how to handle large payloads, integrate external storage, manage asynchronous processing, protect untrusted input, and recover from failures across multiple systems. These lessons apply to media platforms, document applications, reporting systems, and any backend that accepts user-generated files.*
> ***Practice Here***

How to Turn These Projects Into Real Learning

Building all five projects is useful, but the quality of your implementation matters more than the number of repositories on your GitHub profile. A project that contains ten endpoints and no meaningful error handling may teach you less than one carefully designed endpoint that handles concurrency, validation, retries, and failure recovery correctly.

The most effective approach is to build each project in stages. First, implement the smallest complete workflow. Then identify the assumptions that could fail when multiple users, unreliable networks, invalid inputs, or external services enter the picture. Finally, add the design improvements needed to address those specific problems.

Use the following checklist as you work through each project.

- Build the core workflow: Make the primary feature work from request to database and back.
- Validate inputs: Reject malformed data, enforce constraints, and return meaningful errors.
- Handle concurrent requests: Identify where simultaneous operations could corrupt data or produce duplicate results.
- Test failure scenarios: Simulate database errors, external service timeouts, duplicate requests, and interrupted jobs.

- Measure performance: Inspect slow queries, add indexes where justified, and avoid unnecessary database calls.
- Add observability: Use structured logs and track meaningful metrics instead of relying only on console output.
- Deploy the application: Configure environment variables, secure credentials, run migrations where applicable, and document deployment steps.
- Explain your decisions: Record the tradeoffs you made and describe what you would change if traffic or business requirements increased.

Testing deserves particular attention. Unit tests are useful for business rules, while integration tests can verify how the application behaves with its database and other dependencies. For concurrency-sensitive operations, write tests that run competing requests rather than assuming a sequence of isolated requests represents real usage.

Deployment is equally important because it exposes configuration and operational problems that local development may conceal. Learn to manage secrets outside source control, configure production logging, handle database connection limits, and investigate failed deployments. You do not need a complex Kubernetes cluster for every project; a simple deployment with a reliable database and clear documentation can demonstrate sound engineering judgment.

Conclusion: Build Systems That Can Fail, Not Just Features That Work

The biggest improvement in backend development happens when you stop asking whether an endpoint works and start asking what could make it fail. A successful response proves that one request completed under one set of conditions. It does not prove that the system handles concurrent requests correctly, recovers from external service failures, protects user data, or remains maintainable as requirements change.

These five projects offer different ways to develop that mindset. The URL shortener introduces caching and data access patterns, the e-commerce backend explores transactions and consistency, the notification system teaches asynchronous processing, the project management API strengthens authorization and architecture, and the media service reveals the challenges of handling files and external infrastructure. Together, they cover a substantial portion of the problems that backend developers encounter in real applications.

You do not need to implement every advanced feature immediately, and you certainly do not need to turn every project into a distributed system. Start small, understand the behavior of your application, introduce complexity only when a real requirement justifies it, and document the reasoning behind your decisions.

The goal is not to build five impressive projects. It is to become the developer who understands why a system works, where it might break, and how to make it more reliable. That understanding is what tutorials can introduce but hands-on engineering teaches.

> *Below is the complete collection of this kind of stories*

List: Complete Backend Engineer List | Curated by Gopi C K | Medium

Complete Backend Engineer List · 12 stories on Medium

medium.com

Technology + Coding + Programming + Software Development + Backend +



Автор: Гопи Си Кей

1,7 тыс. подписчиков · 26 подписчиков

Пишу о разработке программного обеспечения, программировании, проектировании систем, искусственном интеллекте и уроках, которые я хотел бы усвоить раньше. Подробнее: https://linktr.ee/ckgopi

Подписаться

Поддержка в продвижении идей



Подписаться

Опубликовано в All About Computers

174 подписчика · Последнее обновление 21 час назад

Центр всего, что связано с компьютерами! Здесь вы найдете обучающие материалы, советы, проекты и аналитические материалы по аппаратному и программному обеспечению, операционным системам, программированию, сетевым технологиям, кибербезопасности, искусственному интеллекту и многому другому — все это создано энтузиастами для энтузиастов.